Part 1: Randomized Quicksort

Empirical Comparison Results

Input Size	Input Type	Deterministic Quicksort	Randomized Quicksort
1000	Random	0.001777s	0.001914s
1000	Sorted	Recursion Error	0.001286s
1000	Reverse Sorted	Recursion Error	0.002281s
1000	Repeated	0.005070s	0.000486s
2000	Random	0.003116s	0.004049s
2000	Sorted	Recursion Error	0.002172s
2000	Reverse Sorted	Recursion Error	0.002050s
2000	Repeated	0.011623s	0.000268s
5000	Random	0.007634s	0.012179s
5000	Sorted	Recursion Error	0.008739s
5000	Reverse Sorted	Recursion Error	0.006026s
5000	Repeated	0.077050s	0.000859s

Average-Case Analysis of Randomized Quicksort

Randomized Quicksort chooses a pivot at random and then splits the array into smaller parts. Because the pivot is selected randomly, the algorithm is less likely to keep creating very uneven partitions.

The expected running time can be represented by the recurrence relation:

$T(n)=1n∑k=0n−1[T(k)+T(n−k−1)]+O(n)T(n)=\frac{1}{n}\sum_{k=0}^{n-1}[T(k)+T(n-k-1)] + O(n)$T(n)=n1 k=0∑n−1 [T(k)+T(n−k−1)]+O(n)

where $O(n)O(n)$O(n) is the time needed to partition the array.

On average, the partitions are reasonably balanced, which can be simplified to:

$T(n)=2T(n/2)+O(n)T(n)=2T(n/2)+O(n)$T(n)=2T(n/2)+O(n)

Using the Master Theorem, this gives:

$T(n)=O(n\log n) T(n)=O(n \log n)$T(n)=O(nlogn)

Another way to show this is with indicator random variables. Let $X_{ij} X_{ij}$Xij equal 1 if two elements are compared and 0 otherwise. The probability that two elements are compared is:

$P(X_{ij}=1)=\frac{2}{j-i+1} P(X_{ij}=1)=2j−i+1$P(Xij =1)=j−i+12

When these probabilities are added for all possible pairs of elements, the expected number of comparisons is:

$E[X]=O(n\log n) E[X]=O(n \log n)$E[X]=O(nlogn)

Since comparisons make up most of the work in Quicksort, the expected running time is also:

$\boxed{O(n\log n)} \boxed{O(n \log n)}$O(nlogn)

Comparison and Discussion

The results showed that both algorithms performed similarly on random arrays. For an input size of 5000, Deterministic Quicksort took 0.007634 seconds, while Randomized Quicksort took 0.012179 seconds.

The biggest difference appeared with sorted and reverse-sorted arrays. Deterministic Quicksort produced a RecursionError because using the first element as the pivot created extremely uneven partitions. Randomized Quicksort handled these inputs without any issues, taking 0.008739 seconds on the sorted array and 0.006026 seconds on the reverse-sorted array.

Arrays with repeated values also showed a large difference. For an input size of 5000, Deterministic Quicksort took 0.077050 seconds, while Randomized Quicksort took only 0.000859 seconds. This happened because the three-way partition method handled duplicate values more efficiently.

Overall, the results matched the theoretical analysis. Randomized Quicksort performed well across all input types and supported the conclusion that its expected running time is **O(n log n)**.

Part 2: Hashing with Chaining

Implementation

For this part, I implemented a hash table using chaining to handle collisions. The hash table stores data in buckets, and each bucket can hold multiple key-value pairs if a collision occurs.

The hash function uses Python's built-in hash() function and the modulo operator to determine which bucket a key belongs to.

The hash table supports the following operations:

- **Insert** – adds a new key-value pair to the table.
- **Search** – finds and returns the value associated with a key.
- **Delete** – removes a key-value pair from the table.
- **Display** – shows the contents of all buckets.

To test the implementation, I inserted three records:

- Alice → 90
- Bob → 85
- Charlie → 95

I then searched for Alice and Bob, deleted Bob, and verified that Bob was removed from the table.

Program Output

Plain Text
1
Search Alice: 90
2
Search Bob: 85
3
Search Bob after deletion: None
4

5
Bucket 0: [['Charlie', 95]]

6
Bucket 1: []
7
Bucket 2: []
8
Bucket 3: []
9
Bucket 4: []
10
Bucket 5: []
11
Bucket 6: []
12
Bucket 7: [['Alice', 90]]
13
Bucket 8: []
14
Bucket 9: []
Show more lines

The output shows that Alice and Charlie remained in the hash table, while Bob was successfully removed. The bucket locations may vary between runs because Python's hash values can change.

Analysis

With simple uniform hashing, insert, search, and delete operations have an average time complexity of **O(1)**. This is because the keys are expected to be distributed fairly evenly across the buckets.

In the worst case, many keys could end up in the same bucket. When this happens, the operations may need to search through an entire chain, resulting in a worst-case time complexity of **O(n)**.

Load Factor and Performance

The load factor is calculated using:

$\alpha = \frac{n}{m}$α=mn

where:

- n is the number of stored elements
- m is the number of buckets

A lower load factor means fewer collisions and better performance. As more elements are added, collisions become more common and the chains become longer.

One way to maintain good performance is through **dynamic resizing**. When the load factor becomes too high, the hash table can be expanded and all elements can be rehashed into a larger table. Using a good hash function also helps reduce collisions by spreading keys more evenly across buckets.

Conclusion

This assignment showed how algorithm design affects performance. The results demonstrated that Randomized Quicksort is more reliable than Deterministic Quicksort because it avoids the worst-case behavior caused by poor pivot choices. The hash table implementation also showed how chaining can efficiently handle collisions while maintaining fast insert, search, and delete operations. Overall, both parts highlighted the importance of choosing data structures and algorithms that scale well as the amount of data grows.